

Unnamed project

unnamed project

10.07.2025(DRAFT)

I'm currently teaching myself Braille, which is a new and ongoing process for me. To help draft this book, I'm using the Gemini Help Me Write function. Because I am focusing on capturing my ideas as I learn, you may notice variations in the language and tone across the text. My goal for now is simply to get the content down; I will refine the writing style and consistency once I have the final ideas established.

This free, shareable text file serves as both a **legal study guide** and a [Braille reference dictionary](#). Use it to learn your rights under [Canadian](#), [U.S.](#), and [British constitutional law](#), preparing you for exams like the **Canada bar exam (NCA)**, **UBE** and **SQE1/SQE2**. The included [Perky Duck Braille notation](#) and [Unified English Braille \(UEB\) Grade 1 instructions](#) provide a complete, simple reference. Furthermore, the material is optimized to [generate a custom quiz using Canvas](#), perfect for improving the knowledge of junior reps. Challenge yourself by aiming for a **10/10 score on Level 10** and provide a screenshot for proof!

This document is free to share and I hope it improves the welfare of others. It's my attempt to redeem myself for my past actions.

Stephen Ancliffe

stephen ancliffe



Table of Contents

Introduction: A Guide to Accessibility with Gemini

- [Project Overview:](#) A brief summary of the **Unnamed Project** initiative.
- [Our Goals:](#) The core objectives of the project, including its **Zero-Cost Accessibility** policy.
- [Specifications:](#) Detailed project requirements, including the Zero-Cost Accessibility policy and functions of the Closed Captioning and Braille Generation tools.
- [Warning on Copyright and Gemini-Generated Content:](#) Important warning about the dangers and legal risks of using copyrighted material with Gemini-generated content.
- [What is the Marrakesh Treaty?:](#) Essentially, the Marrakesh Treaty harmonizes copyright law exceptions across the globe to promote the free flow of books and other literary and artistic works in formats that the print-disabled can access.

Part I: Closed Captioning

- [Using Gemini for Closed Captioning:](#) A step-by-step guide for generating subtitles.
- [Troubleshooting & Limitations:](#) Common issues and current barriers in the process.

Part II: Braille Generation & Analysis

- [Establishing a Baseline for Braille Accuracy:](#) Baseline reference for braille.
- [Perky Duck and UEB Unicode grade 1 instructions:](#) Free and easy to set up, this tool breaks down barriers. It's designed to support users who cannot operate a keyboard or traditional hardware. Plus, it offers ultimate convenience: if you forget your usual equipment or software, you can access everything on any standard computer. This is especially helpful for citizens in remote areas who may lack access to specialized software or hardware.
- [Google Translate rough mockup with Braille support:](#) A rough mockup of what a fronted UI would look like. It's nonfunctional at the moment.
- [Dockerized Braille Converter Web Application with didactic-chainsaw:](#) **(Inprogress)** A project for a Dockerized web application that provides offline Braille Grade 1 conversion. The project includes a backend server written in Python with Flask and a simple HTML/CSS/JS frontend.

- [**Braille to Text: Mobile App Concept:**](#) The proposed workflow for a community-driven mobile app.

Part III: Sign language

- [**SignGemma:**](#) SignGemma is a new AI model from Google DeepMind. It translates American Sign Language (ASL) to English text in real-time by analyzing visual cues like hand movements and facial expressions. The model is being developed with community input and is designed to supplement, not replace, human interpreters.

Part IV: Learning

- [**Using Canvas to Generate a Quiz:**](#) This section details how to use Gemini's Canvas feature to create a multiple-choice quiz. It provides step-by-step instructions for selecting the Canvas tool, attaching a PDF of the Canadian Charter of Rights and Freedoms, and using a specific prompt to generate a 10-question quiz with an answer key and hints. The guide also includes tips for adjusting the difficulty level and troubleshooting common issues.

Part V: Unsorted

- [**Enabling Image-to-Speech Analysis for Blind and Low-Vision Users with Google Gemini \(concept\):**](#) Rough mockup of the menu which includes a high contrast option. Web apps may already follow system settings for high contrast. Personally sometimes I just want output in large text for the web application.

Part VI: Your own Artificial Intelligence

- **How to run your own Artificial Intelligence:** Exploring various popular options such as cloud based services and locally hosted options.
- [**Ollama:**](#) What is it and how to use it
- [**AI models focused on specific tasks:**](#) A brief listing of various models and their uses.
- [**How to setup an AI agent with Ollama:**](#) Building a custom AI agent with Ollama is a popular and powerful way to create a private, self-hosted AI assistant.

Appendix

- [Recommended Reading & Resources](#): A list of relevant external links and datasets.

Project Overview

In a world increasingly reliant on digital information, accessibility is not just a feature—it's a necessity. **Unnamed Project** is a forward-thinking initiative designed to ensure that everyone, regardless of their ability, can access and engage with content.

Our project utilizes the advanced capabilities of **Google Gemini** to solve common accessibility challenges. We focus on producing high-quality, specialized content, such as:

- **Closed Captions:** Automatically generating precise and contextually aware closed captions for spoken audio and video, benefiting the Deaf and hard of hearing community.
- **Braille Transcription:** Converting digital text into standardized braille formats, offering a critical resource for individuals who are blind or have low vision.

By transforming digital content into these essential formats, we're not just creating tools—we're creating opportunities. "Echoes & Insights" is committed to building a more equitable digital landscape where information is truly accessible to all.

Our Goals

1. **Goal 1: Maximize Accessibility and Usability.** Develop solutions that are intuitive and require no specialized technical knowledge. The process for generating braille or closed captions should be as simple as inputting text or audio and receiving the accessible output, making the tool usable by individuals, educators, and small organizations alike.
2. **Goal 2: Ensure Financial Accessibility.** The core project tools and services must be entirely free to use, leveraging existing, open-source platforms and APIs (such as Google Gemini's free-tier access) to eliminate cost barriers. The solutions should be designed to function with common, readily available devices, such as a standard computer or smartphone, without the need for expensive, proprietary hardware.
3. **Goal 3: Promote Universal Digital Inclusion.** Provide practical and effective resources that empower individuals with disabilities to independently access digital

content. By generating high-quality braille and closed captions at no cost, the project will directly contribute to a more equitable digital landscape and reduce dependence on costly third-party services.

4. **Goal 4: Build a Self-Sustaining, Community-Driven Resource.** Create a scalable solution that can be improved upon and maintained by a community of users and developers. The project's documentation and code will be made publicly available, encouraging collaboration and ensuring that the tool remains a reliable, free resource for years to come.

Specifications

Zero-Cost Accessibility: The application must be entirely free for all users. It will not require any paid subscriptions, in-app purchases, or specialized hardware. Users will only need a standard web browser on a common device (e.g., a smartphone, laptop, or desktop computer).

Closed Captioning Tool

- **Input:** A file upload button that accepts common audio formats (e.g., MP3, WAV) and video formats (e.g., MP4).
 - **Processing:** The application will use Gemini's existing audio transcription capabilities to generate a structured output that includes both the transcribed text and timestamps.
 - **Output:** The final output will be a downloadable text file, formatted in the industry-standard **.srt** format, making it compatible with most video players and platforms.
-

Braille Generation Tool

- **Input:** A simple text area for users to paste or type plain text.

- **Processing:** The application will leverage Gemini's core text-to-text functionality by prompting the model to convert the plain text into standard **Unicode Braille characters**.
- **Output:** The generated braille will be displayed in a non-editable text box, allowing users to easily copy the content for use with a braille display or embosser software.

Warning on Copyright and Gemini-Generated Content

Using large language models (LLMs) like Gemini to create closed captions for videos can be a helpful tool. However, it's crucial to be aware of the potential for **copyright infringement**.

Gemini may have been trained on copyrighted material, and its output could contain text or creative works that are protected by copyright. This means that if you use Gemini to generate closed captions, the captions may unintentionally replicate copyrighted content.

If you then use or distribute a video with captions that contain copyrighted material, you could be violating the copyright holder's rights. This could expose you to legal risks, including lawsuits and financial penalties.

Gemini currently lacks a feature to filter or identify copyrighted content, so it is your responsibility to ensure that any captions you generate and use are free of copyrighted material. To avoid copyright issues, you should:

- **Review all generated content carefully:** Before using the captions, read them thoroughly to ensure they do not contain copyrighted text.
- **Obtain permission:** If you suspect the content may be copyrighted, you must obtain permission from the copyright holder before using it.
- **Generate original content:** For all videos you create, it is always best to use original text that you have written yourself.

What is the Marrakesh Treaty?

The Marrakesh Treaty (formally, the Marrakesh Treaty to Facilitate Access to Published Works for Persons Who Are Blind, Visually Impaired or Otherwise Print Disabled) is an international treaty adopted by the member states of the World Intellectual Property Organization (WIPO) in 2013.

Its main purpose is a humanitarian one: to address the global "book famine"—the fact that only a tiny fraction of published books are ever made available in formats accessible to people with print disabilities.

Key Provisions and Goals:

Mandatory Copyright Exceptions: It requires all contracting countries to adopt national copyright law exceptions that allow "authorized entities" (like libraries or organizations serving the print-disabled) to produce accessible format copies (like Braille, large print, or audio) of published works without having to seek permission from the copyright holder.

Cross-Border Exchange: Crucially, it establishes rules for the international exchange of these accessible format copies. This means an authorized entity in one member country can legally send an accessible copy of a book to an authorized entity or a person with a print disability in another member country. This eliminates the need for books to be transcribed multiple times in different countries.

Beneficiary Persons: The treaty defines "beneficiary persons" broadly, including those who are:

- Blind.
- Visually impaired or have a perceptual or reading disability (like dyslexia) that prevents them from reading printed works.
- Physically disabled and unable to hold or manipulate a book.

Essentially, the Marrakesh Treaty harmonizes copyright law exceptions across the globe to promote the free flow of books and other literary and artistic works in formats that the print-disabled can access.

Consult the **World Intellectual Property Organization Summary of the Marrakesh Treaty** for this topic.

https://www.wipo.int/treaties/en/ip/marrakesh/summary_marrakesh.html

Consult the relevant **Wikipedia article for further information** on this topic.

https://en.wikipedia.org/wiki/Marrakesh_VIP_Treaty

Consult the **Senate Consideration of Treaty Document 114-6** for this topic

<https://www.congress.gov/treaty-document/114th-congress/6/document-text>

Using Gemini for Closed Captioning

This guide outlines how to use Gemini to create and translate `.srt` closed caption files. An `.srt` file is a simple text file that contains subtitles for a video. Think of it as a script for a movie or show, but with specific timing. It includes:

- **A number:** To keep the subtitles in order.
- **A timestamp:** To tell the video player when to show each line of text and when to make it disappear.
- **The subtitle text:** The actual dialogue or description that appears on the screen.

Because it's just a text file, it's very small and easy to share, and it works with most video players.

Step 1: Create a Closed Caption File

1. **Start a new chat:** Go to Gemini and start a new chat.
2. **Attach the video:** In the chat window, click the plus sign and attach the video file you want to create captions for.
 - **Note on file size:** Each video can be up to 2 GB, with a total video length of up to 5 minutes. Other supported file types can be up to 100 MB. To extend the total upload length to 1 hour, you can upgrade to Google AI Pro or Ultra.
3. **Enter the prompt:** In the chat box, enter the following prompt: "Using the provided video, create an `.srt` closed caption file."
4. **Wait for Gemini:** Allow Gemini to generate the captions.

Step 2: Translate the Caption File (Optional)

1. **Enter the translation prompt:** Once Gemini has generated the captions, enter the following prompt in the same chat box: "convert the `.srt` file output to [Language]." (Remember to replace [Language] with your desired language, such as "French," "Spanish," or "German.")

Step 3: Save and Use the Caption File

1. **Open a text editor:** Open a plain text editor on your computer (like Notepad on Windows or TextEdit on Mac). Do not use a word processor like Microsoft Word, as it may add formatting that can cause problems.

2. **Paste the captions:** Copy the output from Gemini and paste it into the new text file.
3. **Save the file:** Save the file, making sure the file name ends in `.srt`. For example, `en.srt` for English captions or `fr.srt` for French captions.
 - **Ensure UTF-8 encoding:** If available, save the file using "UTF-8" encoding. This helps prevent strange characters from appearing.
 - **On a Mac:** In TextEdit, go to **Format > Make Plain Text**. Then go to **File > Save As**, and select **Unicode (UTF-8)** from the **Plain Text Encoding** dropdown.
 - **On Windows:** In Notepad, go to **File > Save As**. In the **Save As** dialog box, select **All Files** from the "Save as type" dropdown, and then choose **UTF-8** from the "Encoding" dropdown.

Step 4: Add Captions to a Video in VLC

1. **Download and Install VLC:** If you don't already have it, download and install VLC media player from videolan.org.
2. **Open your video:** Launch VLC and open the video file you want to watch.
3. **Add the subtitle file:** In the VLC menu bar, navigate to **Subtitles > Add Subtitle File...**
4. **Select the .srt file:** Browse to where you saved your `.srt` file, select it, and click **Open**. The subtitles should now appear at the bottom of the video.

Troubleshooting & Limitations:

- **Subtitles don't appear:** In VLC, go to the **Subtitles** menu and ensure that the "Subtitles Track" is set to "Track 1" or a similar option.
- **Strange characters:** If you see strange characters, it's likely an encoding issue. Try saving your `.srt` file again with **UTF-8** encoding, following the steps above.

Establishing a Baseline for Braille Accuracy

Creating a baseline to test the accuracy of Unified English Braille (UEB) output involves establishing a set of known, correct transcriptions to which new outputs can be compared. The process is similar to creating a benchmark in software development or machine learning. A simple and effective method is to select a diverse body of text, ensuring it includes a wide range of punctuation, capitalization, numbers, and contractions as defined in the official UEB rules. A human expert, or a trusted and verified transcription service,

Perky Duck and UEB Unicode grade 1 instructions

Free and easy to set up, this tool breaks down barriers. It's designed to support users who cannot operate a keyboard or traditional hardware. Plus, it offers ultimate convenience: if you forget your usual equipment or software, you can access everything on any standard computer. This is especially helpful for citizens in remote areas who may lack access to specialized software or hardware.

Why do I need to use these files?

These simple text-based reference files provide examples for [Gemini's analysis](#). While the current layout is effective, easily printable, and reformattable, the files may contain errors or be missing signs and symbols. I encourage you to make any necessary corrections or additions and share your improved files with the community. This method of using reference files could be extended to other workflows, including Language or Math projects.

Unstructured thoughts: Hmm... you could probably make a dictionary with jargon used in industries and have it translated back and forth for regular terms depending on the audience. I haven't tested this but I'm wondering if it's text based only. What about images (maybe place holders of them)? Math symbols? Medical jargon or short form words? I think I'm just starting to tap into what you can convert. I need to let this idea grow while taking a soak in a tub like [Benjamin Franklin](#) or Archimedes.

Braille Generation Tool Unified English Braille (UEB) Grade 1

Files referenced on Oct 2nd version ([You need to attach this text files to the Gemini conversation for reference](#)):

[braille-dictionary-grade-2.txt](#)

Prompt (manually typed but should work with Gemini's native voice dictation):

Using the attached file convert the sentence below to Unified English Braille (UEB) Grade 1

Stephen Ancliffe

Response:

The sentence "Stephen Ancliffe" converted to Unified English Braille (UEB) Grade 1 (uncontracted braille) is:

.:stphn ancliff

Prompt (manually typed but should work with Gemini's native voice dictation):

Using the attached file convert the sentence below from Unified English Braille (UEB) Grade 1 to English

.:stphn ancliff

Response:

English Conversion

The sentence converted to English is:

Stephen Ancliffe

Braille Generation Tool Perky Duck notation

Files referenced on Oct 2nd version ([You need to attach this text files to the Gemini conversation for reference.](#)):

[braille-dictionary-grade-2-perky-duck.txt](#)

Prompt (manually typed but should work with Gemini's native voice dictation):

Using the attached file convert the sentence below to Perky Duck format

Stephen Ancliffe

Response:

The sentence "Stephen Ancliffe" converted to the Perky Duck six-key chording format, based on the provided UEB Grade 2 dictionary, is:

L DSJ DSJK FK FDSJ FDK FSJK L F FSJK FJ FDS DJ FDJ FDJ FK

Prompt (manually typed but should work with Gemini's native voice dictation):

Using the attached file convert the sentence below from Perky Duck six-key chording format to English

L DSJ DSJK FK FDSJ FDK FSJK L F FSJK FJ FDS DJ FDJ FDJ FK

Response:

English Conversion

The converted sentence is:

Stephen Ancliffe

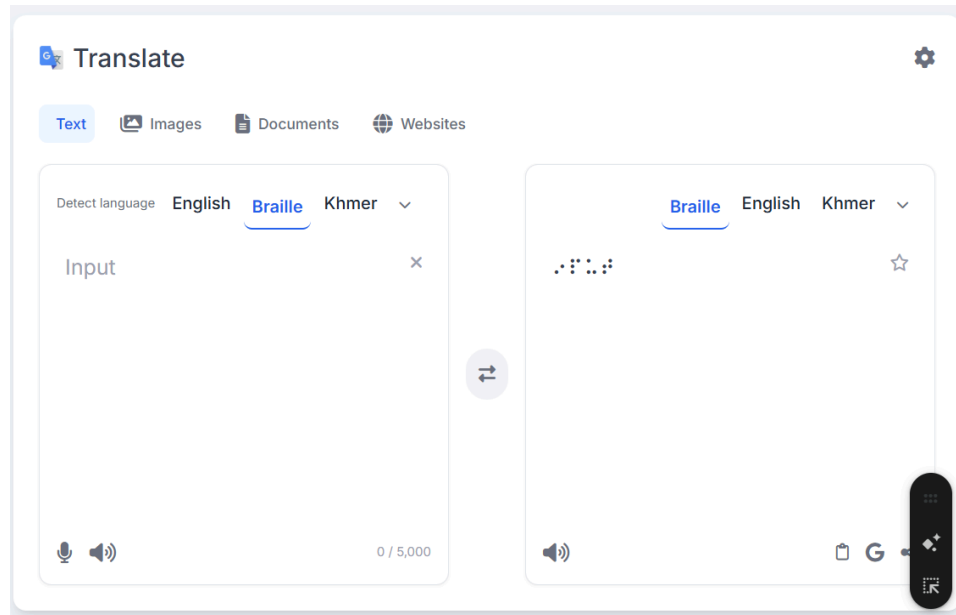
Unanswered questions:

- How much text can I input?
- Will text conversion fail after X amount of characters?
- Can I attach an existing .txt formatted text file and do bulk conversions to Perky Duck format?
- How large can files be when you convert using the free tier of Gemini? (**Equitable access to all**)
- Can you convert an .srt to .brf so the blind can use a reader while the movie plays? (Is that even a thing? Do blind people use any special equipment when going to the movie theater? I need a consultation on this.)

Dockerized Braille Converter Web Application with didactic-chainsaw

The project's source code now includes **multiple translation files** within the [GitHub](#) repository, significantly expanding its capabilities for converting UEB Grade 1 braille. While the underlying code is robust, a key focus of ongoing development remains on creating a **simple, easily distributable application** for end-users. A promising solution under consideration is an **offline-capable HTML and JavaScript interface**. This approach would allow users to simply open the file in any modern web browser, enabling drag-and-drop translation functionality without requiring an active internet connection or a complex installation process.

near-instantaneous translation experience that could render foreign text directly into a tactile Braille output. This foundational work wasn't just about text conversion; it required developing a novel interface where the complex linguistic structures managed by the AI could be accurately and quickly digitized for physical presentation to the user.



SignGemma

SignGemma is a new AI model from Google DeepMind that is designed to translate American Sign Language (ASL) into English text in real-time. It's a "multimodal" model, which means it can process both video and language, and it is trained to understand not just hand movements, but also facial expressions and body posture, which are critical parts of ASL grammar and nuance.

The model is part of Google's broader "Gemma" family of open-weight models, and its development has a strong focus on community collaboration, with input from developers and members of the Deaf community. It is designed to run on-device, like on smartphones or laptops, which helps preserve user privacy by processing video data locally.

SignGemma is intended to serve as a bridge for communication between Deaf and hearing individuals and is seen as a significant step toward making digital communication more inclusive. It is not meant to replace human interpreters but rather to supplement them in a variety of settings. While currently focused on ASL, the project's open-source nature suggests potential for future expansion to other sign languages.

Deaf News: Interview with Google's Sam Sepah on SignGemma, an upcoming ASL-to-text AI model

<https://www.youtube.com/watch?v=ZB4JoXbLwMo>

Google Invites Feedback for SignGemma, a New AI Sign Language Translation Model

<https://slator.com/google-invites-feedback-for-signgemma-a-new-ai-sign-language-translation-model/>

Image-to-Braille: Current Limitations & Future Solutions

The current implementation of image-to-Braille conversion faces several significant challenges. These limitations are primarily due to the inherent complexities of visual data and the current state of AI technology.

- **Image Processing Difficulties:** The core of image-to-Braille is a process known as Optical Braille Recognition (OBR). This is more challenging than traditional Optical Character Recognition (OCR) because it must accurately identify and interpret the raised dots on a surface, rather than continuous strokes. Factors like poor image quality, glare, shadows, and the need to process images from different angles (e.g., from a phone camera vs. a flatbed scanner) can lead to inaccuracies.
- **AI Hallucination:** A key challenge is the potential for AI models to "hallucinate" or generate incorrect information. This can happen when the model encounters low-quality or ambiguous images, leading it to output Braille characters that do not exist in the source image. This problem is particularly relevant for images of complex content, such as STEM (science, technology, engineering, and mathematics) diagrams or charts, which are not well-represented in existing training data.
- **Multilingual Support:** Most current OBR systems are not designed for multilingual conversion. Each language has its own unique Braille system, and a single model trained on one language may not be able to accurately translate another. This is a critical area for future development to ensure the tool is globally useful.
- **Lack of Context:** An AI model may be able to transcribe the Braille dots it "sees," but it cannot interpret the broader context of the image. For instance, it can't distinguish between a decorative image and one that contains vital information.

Recommendations for Future Development

To address these limitations, consider the following improvements in future versions of the project:

- **Data Quality and Curation:** To mitigate AI hallucination and improve accuracy, a primary focus should be on building a high-quality, diverse dataset of Braille images. This dataset should include images from different angles, lighting conditions, and languages, and it should specifically contain examples of complex diagrams and charts.
- **Enhanced Image Pre-processing:** Implementing more robust image processing algorithms can help to clean up source images before they are passed to the AI model. This could include functions for correcting image skew, enhancing contrast, and removing shadows or glare.
- **Iterative Human-in-the-Loop Model:** Given the current limitations, the most reliable approach is to implement a human review process. This "human-in-the-loop" system would allow users to review and correct the AI's Braille transcription, and this corrected data could then be used to further train and refine the model.
- **Modular Architecture:** The system could be designed with a modular architecture that allows for the integration of specialized models for different tasks, such as a model specifically for STEM Braille, and another for recognizing different languages.

Braille to Text: Mobile App Concept (Need to get a new phone)

Ever since I broke my Android and switched to a [MAXWEST Vice 4G Flip Phone](#), I've found a surprising benefit: a significant increase in free time. I'm no longer stuck "[doomscrolling](#)." Despite the appeal of that extra time, I do miss the functionality of a smartphone, especially email and web browsing. Since I'm currently out of work and need to save money, I'm hesitant to make the purchase, but I recognize the need to buy one.

This document outlines the high-level steps and technical considerations for building a mobile application that translates a sheet of Braille into plain text using a mobile device's camera and a large language model with vision capabilities, such as the Gemini API.

1. Image Capture from a Mobile Device

This initial step is about getting a clear, high-quality image of the Braille text. The quality of this image is critical for the entire process.

- **User Interface (UI):** Provide a camera view with clear on-screen guidance. A bounding box or an outline could help the user frame the Braille sheet correctly.
- **Camera Configuration:** The app should automatically adjust for optimal settings.
 - **Focus:** Implement auto-focus on the Braille dots.
 - **Lighting:** Detect poor lighting conditions and suggest a brighter environment or enable the flash.
 - **Image Stabilization:** Use the device's built-in stabilization to prevent blur from camera shake.
- **Permissions:** Request and verify camera access permissions from the user.

2. Local Image Pre-processing

Once the image is captured, it should be processed on the device to optimize it for transfer and analysis by the model.

- **Cropping & Alignment:** Automatically detect the Braille sheet within the image and crop any unnecessary background. Apply a perspective correction algorithm to align the image so the Braille is straight and not at an angle.
- **Grayscale Conversion:** Convert the image from color to grayscale. This can reduce file size and simplify the image for the model, as Braille dot information is based on contrast, not color.
- **Compression:** Compress the image to a suitable format (e.g., JPEG with a lower quality setting) and a manageable file size for efficient network transfer. This is a critical step, especially for users on mobile data networks.
- **Image Format:** The final image should be a Base64-encoded string for inclusion in the API request payload.

3. Transfer to Gemini API

The prepared image is now ready to be sent to the Gemini API for analysis.

- **API Endpoint:** Make a **POST** request to a Gemini API endpoint capable of multimodal understanding. The **gemini-2.5-flash-preview-05-20** model is a suitable choice for this task.

- **Payload Construction:** The request payload will contain the user's prompt (e.g., "Translate the Braille in this image into English text.") and the Base64-encoded image data.
- **Loading State:** Implement a clear loading indicator on the UI to inform the user that the image is being processed.
- **Error Handling:** Include robust error handling for network failures or API-side errors, providing a user-friendly message if the request fails.

4. Gemini Model Processing & Translation

This is the core of the concept, where the AI performs the heavy lifting. The app's logic ends here, but it's important to understand what the model does.

- **Image Analysis:** The Gemini model analyzes the image, recognizing the Braille dots and their arrangement.
- **Braille Decoding:** The model understands the six-dot Braille system and decodes the patterns into individual Braille characters.
- **Text Translation:** The model translates the decoded Braille characters into the corresponding text in the desired language (e.g., English, Spanish, etc.). It can also handle variations like Grade 1 (letter-for-letter) and Grade 2 (with contractions) Braille.

5. Receive and Process Results

Once the Gemini API completes its analysis, the mobile app receives the response.

- **JSON Response:** The API will return a JSON object containing the translated text.
- **Data Extraction:** The app must parse the JSON response and extract the translated text from the designated field.

6. Display Results & User Interaction

The final step is to present the translated text to the user in an accessible and helpful way.

- **UI Display:** Show the translated text in a clean, legible format on the screen.
- **Accessibility Features:**
 - **Text-to-Speech (TTS):** Add a button to read the translated text aloud for visually impaired users.

- **Copy to Clipboard:** Provide a quick way to copy the translated text.
- **Font Size Adjustment:** Allow the user to adjust the font size for better readability.
- **Feedback:** The UI should transition from the loading state back to a result view, providing clear confirmation that the translation is complete.

Existing github projects which are working on braille to text conversion.

AEyeAlliance (Canadian effort AI4SocialGood Lab 2018 at McGill University. They are doing some wonderful things.)

wanchunsu Wan-Chun Su

tingyisu Ting-Yi Su

HelenGezahegn Helen Gezahegn

marin0829

<https://github.com/AEyeAlliance/AEyeAlliance>

AaditT/braille

Aadit Trivedi

<https://github.com/AaditT/braille>

Braille to Text Translator

Bárbara Côrtes e Souza (@barbaracortes)

Lucas Fernandes Turci (@lucasturci)

<https://github.com/lucasturci/BrailleTextTranslator>

InSight

SatyamSoni23

<https://github.com/SatyamSoni23/InSight>

Using Canvas to Generate a Quiz

To start using **Canvas** in **Gemini**, click the **Tools** icon in the conversation window. For this example we will be using the Canadian Charter of Rights and Freedoms. You can download the PDF and attach the file in the URL listed below for example.

English

<https://www.canada.ca/content/dam/pch/documents/services/download-order-charter-bill/canadian-charter-rights-freedoms-eng.pdf>

French

<https://www.canada.ca/content/dam/pch/documents/services/download-order-charter-bill/charte-canadienne-droits-libertes-fra.pdf>

**This quiz is based on content from the Canadian Charter of Rights and Freedoms, which is subject to Crown copyright*

Steps to add a file from Google Help Center

[Upload & analyze files in Gemini Apps](#)

From there, select **Canvas**. Once **Canvas** is open, you can use either the English or French the prompt below based on which PDF you downloaded and attached.

English prompt:

"Create a 10-question exam with an answer key based on the 'canadian-charter-rights-freedoms-eng.pdf' document. The questions should be at a difficulty level of 7, requiring advanced comprehension and analysis of the text. Each question must have four multiple-choice options and a hint."

Google Gemini [Gem](#) for English (Inprogress. This avoids having to download and attach a PDF file which may be blocked due to security reasons. I also like how Google implemented prompt sharing in their eco-system. It's a unique feature. Google also does an excellent job of scanning attachments used for harmful content (Virus, malware). Very good solution for an office place and easy to deploy.)

<https://gemini.google.com/gem/1XMZgidDH7yeaMUodMm-SDdRZalnjYK3i?usp=sharing>

Invite en français:

"Créez un examen de 10 questions avec un corrigé basé sur le document « charte-canadienne-droits-libertes-fra.pdf ». Les questions doivent avoir un niveau de difficulté de 7 et exiger une compréhension et une analyse approfondies du texte. Chaque question doit comporter quatre choix multiples et un indice."

Google Gemini [Gem](#) pour le français (en cours de développement. Cela évite de télécharger et de joindre un fichier PDF, qui pourrait être bloqué pour des raisons de sécurité.

J'apprécie également la façon dont Google a intégré le partage rapide dans son écosystème. C'est une fonctionnalité unique. Google effectue également un excellent

travail d'analyse des pièces jointes pour détecter les contenus nuisibles (virus, logiciels malveillants). Excellente solution pour un bureau, facile à déployer.)

<https://gemini.google.com/gem/1hmpl26cSIjVXxcOb7q0U-1Th3nMk7dOH?usp=sharing>

English focus on section example:

"Create a 10-question exam with an answer key based on the 'canadian-charter-rights-freedoms-eng.pdf' document. The questions should be at a difficulty level of 7, requiring advanced comprehension and analysis of the text. Each question must have four multiple-choice options and a hint. Focus on section 15.(1) of the document."

Canvas sample level 7

<https://g.co/gemini/share/fb325546dc73>

Canvas sample level 10

<https://g.co/gemini/share/76c5c72a41f4>

Exemple de section de concentration en français :

"Créez un examen de 10 questions avec un corrigé basé sur le document 'charte-canadienne-droits-libertes-fra.pdf'. Les questions doivent avoir un niveau de difficulté de 7 et exiger une compréhension et une analyse approfondies du texte. Chaque question doit comporter quatre choix multiples et un indice. Concentrez-vous sur la section 15.(1) du document."

Exemple de toile niveau 7

<https://g.co/gemini/share/25984683ca1a>

Échantillon de toile niveau 10

<https://g.co/gemini/share/357698c3432f>

Tips for Generating a Quiz

- To achieve the best results, begin with a difficulty level of 7. Gemini may need time to process the material and might refuse a higher difficulty (9 or 10) on the first try.
- To gradually increase the difficulty, start by using the prompt **"generate another quiz."** After running this prompt several times, switch to **"generate a level 10 quiz."**

You don't need to complete every quiz; simply wait for each one to be fully generated before entering the prompt again. This repeated generation process helps Gemini build its knowledge of the document and conduct further research.

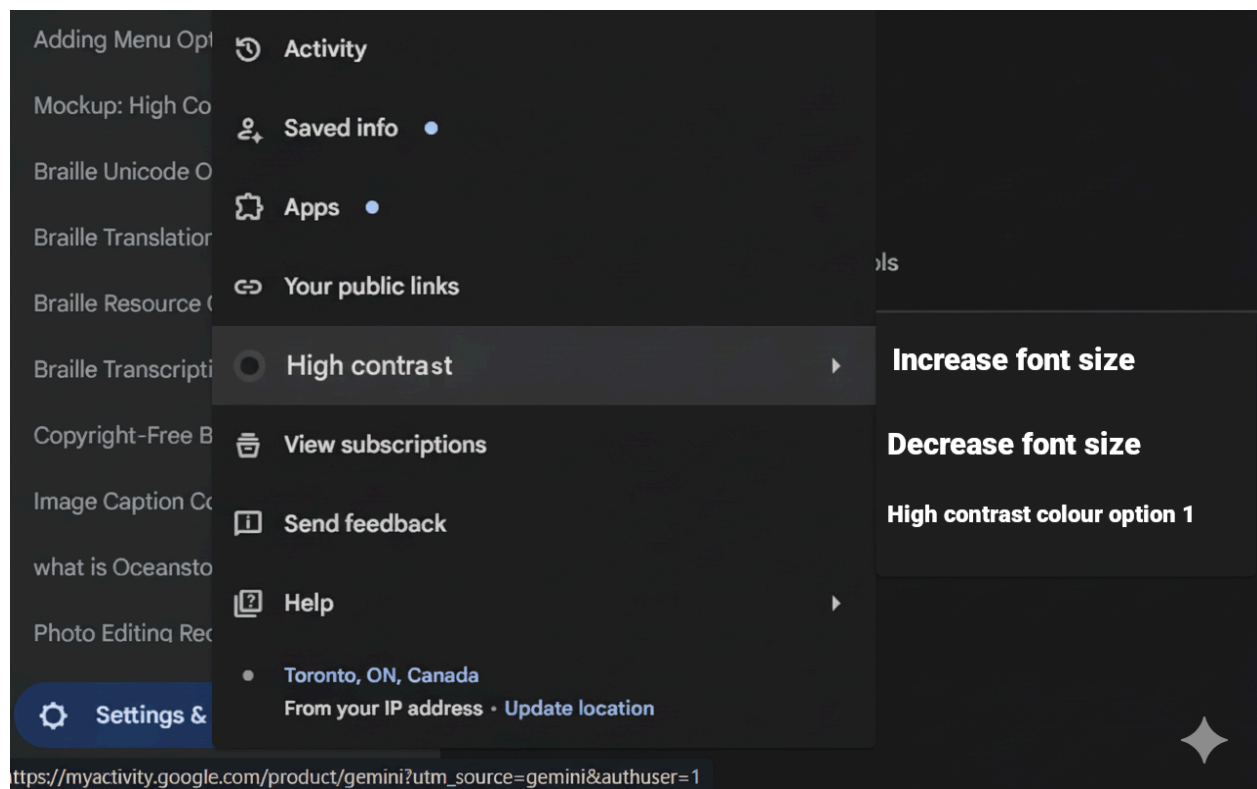
- For now, try to stick with a **maximum of 10** questions to avoid formatting issues. Gemini also doesn't yet support a mix of multiple-choice and true/false questions.
- You can use a prompt like "**show your work for question 1**" to see how Gemini arrived at an answer, which can be particularly useful for more difficult questions (level 9 or 10) where you might disagree with the result.
- If you get an answer wrong, you can ask Gemini to explain the topic and generate 10 more questions to test your understanding.
- Gemini does not track the number of quizzes you've generated, so you'll need to keep count yourself.
- If you're happy with a quiz, click the **Good Response** icon to give Gemini helpful feedback.

Future ideas

- Export graded results to Google Sheets to easily track student progress. An AI assistant can help you format the data in Google Sheets to create detailed grading reports.
- How do I add the following to a quiz:
 1. True or false questions
 2. A chart, bar or pie graph
 3. Multiple choice question based on the picture shown (*which signage is correct?*)
 4. Multimedia video on Youtube (*Watch the video and choose the right multiple choice option. Include YouTube URL when creating a Gem. Disable Gem if YouTube video is modified for security to prevent replacing existing YouTube videos with offensive content, advertising, unauthorized content swap.*)
 5. Written exam questions (*The results and AI grading on this one would probably be disputed frequently.*)

Enabling Image-to-Speech Analysis for Blind and Low-Vision Users with Google Gemini (concept)

A proposed application of Google's Gemini is Image-to-Speech Analysis, designed to empower Blind and Low-Vision users. This tool would bridge the visual gap by describing images verbally. Crucially, the user experience is built around accessibility: a high-contrast mode is included in the menu mockup, although web applications often inherit system-wide settings. Furthermore, to accommodate varying degrees of sight, the interface prioritizes a large-text output option specifically within the application.



Ollama

Ollama's official documents can be found [here](#).

Checking the Ollama service status using Systemd on Linux.

```
:~$ systemctl status ollama
```


● ollama.service - Ollama Service

Loaded: loaded (/etc/systemd/system/ollama.service; enabled; preset: disabled)

Drop-In: /usr/lib/systemd/system/service.d

└─10-timeout-abort.conf

Active: active (running) since Fri 2025-09-26 10:28:24 EDT; 37min ago

Invocation: 3289082be53948b5b3a1377719c2dca4

Main PID: 34551 (ollama)

Tasks: 18 (limit: 18602)

Memory: 4.5G (peak: 9.1G)

CPU: 17min 51.894s

CGroup: /system.slice/ollama.service

└─34551 /usr/local/bin/ollama serve

Sep 26 10:49:48 slate ollama[34551]: time=2025-09-26T10:49:48.185-04:00 level=INFO
source=server.go:1289 msg="llama runner started in 2.03 seconds"

Sep 26 10:49:48 slate ollama[34551]: [GIN] 2025/09/26 - 10:49:48 | 200 | 2.536915677s |
127.0.0.1 | POST "/api/generate"

Sep 26 10:50:33 slate ollama[34551]: [GIN] 2025/09/26 - 10:50:33 | 200 | 29.289452996s |
127.0.0.1 | POST "/api/chat"

Sep 26 10:51:20 slate ollama[34551]: [GIN] 2025/09/26 - 10:51:20 | 200 | 36.367720936s |
127.0.0.1 | POST "/api/chat"

Sep 26 10:54:27 slate ollama[34551]: [GIN] 2025/09/26 - 10:54:27 | 200 | 2m52s |
127.0.0.1 | POST "/api/chat"

Sep 26 10:54:37 slate ollama[34551]: [GIN] 2025/09/26 - 10:54:37 | 200 | 48.267µs | 127.0.0.1 | HEAD "/"

Sep 26 10:54:37 slate ollama[34551]: [GIN] 2025/09/26 - 10:54:37 | 200 | 165.648226ms | 127.0.0.1 | POST "/api/show"

Sep 26 10:54:37 slate ollama[34551]: [GIN] 2025/09/26 - 10:54:37 | 200 | 222.913828ms | 127.0.0.1 | POST "/api/generate"

Sep 26 10:56:26 slate ollama[34551]: [GIN] 2025/09/26 - 10:56:26 | 200 | 1m34s | 127.0.0.1 | POST "/api/chat"

Sep 26 10:58:16 slate ollama[34551]: [GIN] 2025/09/26 - 10:58:16 | 200 | 1m4s | 127.0.0.1 | POST "/api/chat"

Gemini Analysis of Output Data

It looks like you're running the **Ollama** service, which is a tool for running large language models locally. The output you provided gives a detailed status of the service, its resource usage, and a log of recent interactions.

Here is an analysis of the key information in the `systemctl status ollama` output:

Ollama Service Status Analysis ⚙️

Field	Value	Interpretation
Active	active (running)	The Ollama service is currently running successfully.
Uptime	37min ago	The service started 37 minutes ago (at 10:28:24 EDT).

Main PID	34551	The process ID of the main Ollama server process.
Memory	4.5G (peak: 9.1G)	The service is currently using 4.5 Gigabytes of RAM . It peaked at 9.1 GB, likely when a large model was initially loaded or during a particularly long generation. This high usage is typical for running large language models.
CPU	17min 51.894s	This is the total CPU time consumed by the service since it started. It indicates the process has been actively working (performing inference or loading models) for a significant portion of its uptime.
Process	ollama serve	The service is running the Ollama server process, which manages the models and handles API requests.

```
~/git-projects/didactic-chainsaw$ ollama run gemma3:4b
```

```
>>> hello
```

```
Hello there! How can I help you today? 😊
```

Successful real-world deployments of AI solutions.

```
:~$ ollama pull crown/lawx
```

```
pulling manifest
```

```
pulling 00e1317cbf74: 100%
```



pulling 4fa551d4f938: 100%



pulling c608dc615584: 100%



pulling be6abdf03e56: 100%



pulling d47ab88b61ba: 100%



pulling 4eacfe09e3e5: 100%



verifying sha256 digest

writing manifest

success

AI models focused on specific tasks

Inprogress

Case Study: Ollama Models for the Legal Domain

- initium/law_model
- ALIENTELLIGENCE/attorney
- ALIENTELLIGENCE/contractanalyzer
- crown/lawx

Broadening the Scope: General and Multipurpose Ollama Models

- **Foundation Models:** Llama 3 / Llama 3.1 (8B), Mistral (7B), Phi-3
- **Specialized Utility Models:** Magistral, Embeddinggemma, Nomic-embed-text

How to setup an AI agent with Ollama

Inprogress

Building a custom AI agent with Ollama is a popular and powerful way to create a private, self-hosted AI assistant. The most common and robust method involves using a framework like **LangChain** or **CrewAI** in Python, as these frameworks handle the complex logic of agent operation.

Here is a step-by-step guide on the general process:

1. Core Prerequisites

Before building your agent, you need the foundational tools installed and running.

1. Install Ollama:

- Download and install Ollama for your operating system (macOS, Windows, or Linux) from the official website.
- **Run the Ollama server** in the background, usually with the command `ollama serve` in your terminal, or it might run automatically as a service.

2. Pull an LLM:

- Choose a model from the [Ollama library](#) that is suitable for agents. Models that support **tool calling** are highly recommended for the best agent performance.
- **Example:** Pull a tool-calling-capable model like Qwen or Llama 3.1:
- Bash

```
ollama pull qwen2.5
```

OR

```
ollama pull llama3.1
```

○

3. Set up your Python Environment:

- Create and activate a virtual environment.
- Install the necessary libraries, at minimum, a framework and the Ollama integration:
- Bash

```
pip install langchain langchain-ollama
```

OR

```
pip install crewai crewai-tools
```

○

2. Agent Construction with a Framework (e.g., LangChain)

LangChain provides the components needed to define the Large Language Model (LLM), the tools (functions the agent can use), and the logic that connects them.

Step-by-Step Example (Conceptual)

Step 1: Define Custom Tools

An agent's power comes from its tools. Tools are simple Python functions with detailed docstrings, which the LLM uses to decide when and how to call them.

Python

```
from langchain.agents import tool

import datetime
```

@tool

```
def get_current_datetime(format: str = "%Y-%m-%d %H:%M:%S") -> str:
```

```
    """
```

```
    Returns the current date and time, formatted according to the provided
    Python strftime format string. Use this tool whenever the user asks for
    the current date, time, or both.
```

```
    Example format: '%Y-%m-%d' for date, '%H:%M:%S' for time.
```

```
    """
```

```
    return datetime.datetime.now().strftime(format)
```

```
tools = [get_current_datetime]
```

Step 2: Initialize the Ollama Model

You need to load the local Ollama model (e.g., [llama3.1](#)) into your framework.

Python

```
from langchain_ollama import ChatOllama
```

```
# Ensure Ollama server is running (ollama serve)
```

```
llm = ChatOllama(
```

```
model="llama3.1",  
temperature=0, # Lower temperature for more predictable tool use  
)
```

Bind the tools to the LLM. This is what allows the LLM to know about them.

```
llm_with_tools = llm.bind_tools(tools)
```

Step 3: Define the Agent Logic and Prompt

The agent logic combines a **Prompt Template**, the **LLM**, and the **Tools** into an executable entity. The prompt is crucial for giving your agent its "personality" and instructions.

Python

```
from langchain_core.prompts import ChatPromptTemplate  
from langchain.agents import AgentExecutor  
from langchain.agents.format_scratchpad.openai_tools import  
format_to_openai_tool_messages  
from langchain.agents.output_parsers.openai_tools import OpenAIToolsAgentOutputParser  
from langchain import hub # For a pre-built agent prompt
```

Pull a ReAct-style prompt from LangChain Hub (often works well for tool-calling)

```
prompt = hub.pull("hwchase17/openai-tools-agent")
```

Create the agent chain

```
agent = (  
    {  
        "input": lambda x: x["input"],  
        "agent_scratchpad": lambda x: format_to_openai_tool_messages(x["intermediate_steps"]),  
    }  
    | prompt  
    | llm_with_tools  
    | OpenAIToolsAgentOutputParser()  
)
```

```
# Create the final executor
```

```
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

Step 4: Run the Agent

Now you can invoke the agent with a query.

Python

```
# Example run
```

```
response = agent_executor.invoke({"input": "What is the current date and time in the format 'Year-Month-Day'?"})
```

```
print(response["output"])
```

Alternative Frameworks

- **CrewAI / AutoGen:** These frameworks are excellent for building **multi-agent systems**, where different agents with different skills (e.g., a "Researcher" and a "Writer") collaborate to solve a complex task. You can use the [OllamaLLM](#) component as the local backend for each agent.
- **Langflow / Open WebUI:** These offer a low-code or web interface for building and interacting with your local Ollama agents without needing to write code from scratch.

3. Advanced Customization: Modelfiles

You can customize the base LLM itself for your agent's specific role using an **Ollama Modelfile**. This allows you to set a permanent system prompt and other model parameters.

Modelfile Example

Create a file named [Modelfile](#):

```
FROM mistral:7b
```

```
SYSTEM You are a friendly, highly concise financial analyst named 'Ollie'. Your primary role is to answer questions about stock market trends using your tools. Do not answer questions outside of finance.
```

```
PARAMETER temperature 0.2
```

```
PARAMETER top_k 40
```

Create the Custom Model

1. Create the new model based on your Modelfile:
2. Bash `ollama create ollie-finance -f ./Modelfile`

3. Now you can use [ollie-finance](#) in your Python agent script as the model name.

Recommended Reading & Resources

Prompt Engineering and Effective AI Communication

- **Prompting Guide:** [Gemini for Google Workspace Prompting Guide 101](#) This guide provides foundational skills for writing effective prompts, including using natural language, being clear and concise, and providing context. It also covers best practices like breaking down complex tasks and provides examples for various roles like project managers and marketing specialists.
- **The Next AI Career:** [Mark Cuban's Perspective on AI](#) Mark Cuban suggests that the next big career in AI isn't coding, but rather knowing how to effectively use AI tools. This emphasizes the importance of skills like prompt engineering and understanding how to apply AI to solve real-world problems.
- **Google Cloud Startup technical guide AI agents:** Download it [here](#). Are you ready to build your AI agent? The Startup technical guide: AI agents, provides the knowledge needed to go from an idea to prototype to scale, whether your goals are to automate tasks, enhance creativity, or launch entirely new user experiences for your startup.
- **Fortune article on how China is now offering AI to 6-year-olds:** Learning to use artificial intelligence is no longer optional for today's students—it's essential. Children who don't learn these skills will be competing in a future job market against peers who are already experts.
- **Generative AI in Google Workspace Privacy Hub:** The document "Generative AI in Google Workspace Privacy Hub" explains Google's privacy and data handling practices for its generative AI features, including Gemini in Workspace Apps. It emphasizes that Google maintains its existing privacy commitments, keeping user data and interactions within their organization. The article details that user content is not used to train AI models without explicit permission, and provides a summary of data practices, security controls, and compliance with regulations like HIPAA and GDPR. It also covers features like Google grounding and user/admin controls.

Accessibility and Alternative Communication

- **How to read and write Braille:** [How to read and write Braille?](#) This video explains the Braille script, which uses a system of six dots. It demonstrates how to write

using a Braille slate and stylus, and explains that writing is done from right to left, while reading is done from left to right. It also goes over the dot combinations for each letter of the English alphabet.

Gemini Apps limits & upgrades for Google AI subscribers

	Gemini <i>without a Google AI plan</i>	Gemini app in Google AI Pro	Gemini app in Google AI Ultra
<u>Gemini Models</u>			
2.5 Pro	Up to 5 prompts / day	Up to 100 prompts / day	Up to 500 prompts / day
2.5 Flash	General access		
<u>Context Window</u>			
Context size	32 thousand	1 million	
<u>Features</u>			
Audio Overviews	Up to 20 audio overviews / day		
Deep Research	Up to 5 reports / month using 2.5 Flash	Up to 20 reports / day using 2.5 Pro	Up to 200 reports / day using 2.5 Pro
Deep Think	-	-	Up to 10 prompts / day with a 192 thousand token context window

Image generation & editing	Up to 30 images / day	Up to 1,000 images / day	
Scheduled actions	-	Up to 10 active actions at a time	
Video generation	-	Up to 3 videos / day using Veo 3 Fast (preview)	Up to 5 videos / day using Veo 3 (preview)
Additional features*	Usage limits subject to the selected model		
Early access to features	-	Priority access to some new features	

*Tip: There are more features that are not listed above, like Canvas, Gems, and more. These features are generally available to most users. [Learn more in the Help Center](#).

Why did I choose Gemini instead of other AI models? Good integration with existing products. Works with **YouTube, Google Docs, Gmail, Google Maps** and other products in the Google eco system I frequently use.

The original is a short, unpolished transcript, which is not from a high quality source. It is created with help from Google AI to make it more readable and to help with the overall quality of the transcript.